

Software Testing Principles

Vital Program Testing Guidelines	
Principle Number	Principle
1	A necessary part of a test case is a definition of the expected output or result.
2	A programmer should avoid attempting to test his or her own program.
3	A programming organization should not test its own programs.
4	Thoroughly inspect the results of each test.
5	Test cases must be written for input conditions that are invalid and unexpected, as well as for those that are valid and expected.
6	Examining a program to see if it does not do what it is supposed to do is only half the battle; the other half is seeing whether the program does what it is not supposed to do.
7	Avoid throwaway test cases unless the program is truly a throwaway program.
8	Do not plan a testing effort under the tacit assumption that no errors will be found.
9	The probability of the existence of more errors in a section of a program is proportional to the number of errors already found in that section.
10	Testing is an extremely creative and intellectually challenging task.

Principle 1: A necessary part of a test case is a definition of the expected output or result.

This obvious principle is one of the most frequent mistakes in program testing. Again, it is something that is based on human psychology. If the expected result of a test case has not been predefined, chances are that a plausible, but erroneous, result will be interpreted as a correct result because of the phenomenon of “the eye seeing what it wants to see.” In other words, in spite of the proper destructive

definition of testing, there is still a subconscious desire to see the correct result. One way of combating this is to encourage a detailed examination of all output by precisely spelling out, in advance, the expected output of the program. Therefore, a test case must consist of two components:

1. A description of the input data to the program.
2. A precise description of the correct output of the program for that set of input data.

A problem may be characterized as a fact or group of facts for which we have no acceptable explanation, that seem unusual, or that fail to fit in with our expectations or preconceptions. It should be obvious that some prior beliefs are required if anything is to appear problematic. If there are no expectations, there can be no surprises.

Principle 2: A programmer should avoid attempting to test his or her own program.

Any writer knows—or should know—that it's a bad idea to attempt to edit or proofread his or her own work. You know what the piece is *supposed* to say and may not recognize when it says otherwise. And you really don't want to find errors in your own work. The same applies to software authors.

Another problem arises with a change in focus on a software project. After a programmer has *constructively* designed and coded a program, it is extremely difficult to suddenly change perspective to look at the program with a *destructive* eye.

As many homeowners know, removing wallpaper (a destructive process) is not easy, but it is almost unbearably depressing if it was your hands that hung the paper in the first place. Similarly, most programmers cannot effectively test their own programs because they cannot bring themselves to shift mental gears to attempt to expose errors. In addition, a programmer may subconsciously avoid finding errors for fear of retribution from peers or from a supervisor, a client, or the owner of the program or system being developed.

In addition to these psychological issues, there is a second significant problem: The program may contain errors due to the programmer's misunderstanding of the

problem statement or specification. If this is the case, it is likely that the programmer will carry the same misunderstanding into tests of his or her own program.

This does not mean that it is impossible for a programmer to test his or her own program. Rather, it implies that testing is more effective and successful if someone else does it.

Note that this argument does not apply to debugging (correcting known errors); debugging is more efficiently performed by the original programmer.

Principle 3: A programming organization should not test its own programs.

The argument here is similar to the previous argument. A project or programming organization is, in many senses, a living organization with psychological problems similar to those of individual programmers. Furthermore, in most environments, a programming organization or a project manager is largely measured on the ability to produce a program by a given date and for a certain cost. One reason for this is that it is easy to measure time and cost objectives, but it is extremely difficult to quantify the reliability of a program. Therefore, it is difficult for a programming organization to be objective in testing its own programs, because the testing process, if approached with the proper definition, may be viewed as decreasing the probability of meeting the schedule and the cost objectives.

Again, this does not say that it is *impossible* for a programming organization to find some of its errors, because organizations do accomplish this with some degree of success. Rather, it implies that it is more economical for testing to be performed by an objective, independent party.

Principle 4: Thoroughly inspect the results of each test.

This is probably the most obvious principle, but again it is something that is often overlooked. We've seen numerous experiments that show many subjects failed to detect certain errors, even when symptoms of those errors were clearly observable on the output listings. Put another way, errors that are found on later tests are often missed in the results from earlier tests.

Principle 5: Test cases must be written for input conditions that are invalid and unexpected, as well as for those that are valid and expected.

There is a natural tendency when testing a program to concentrate on the valid and expected input conditions, at the neglect of the invalid and unexpected conditions.

Few people, for instance, feed the program the numbers 1, 2, 5 to make sure that the program does not erroneously interpret this as a scalene triangle instead of an invalid triangle. Also, many errors that are suddenly discovered in production programs turn up when the program is used in some new or unexpected way. Therefore, test cases representing unexpected and invalid input conditions seem to have a higher error-detection yield than do test cases for valid input conditions.

Principle 6: Examining a program to see if it does not do what it is supposed to do is only half the battle; the other half is seeing whether the program does what it is not supposed to do.

This is a corollary to the previous principle. Programs must be examined for unwanted side effects. For instance, a payroll program that produces the correct paychecks is still an erroneous program if it also produces extra checks for nonexistent employees or if it over- writes the first record of the personnel file.

Principle 7: Avoid throwaway test cases unless the program is truly a throwaway program.

This problem is seen most often with interactive systems to test programs. A common practice is to sit at a terminal and invent test cases on the fly, and then send these test cases through the program. The major problem is that test cases represent a valuable investment that, in this environment, disappears after the testing has been completed. Whenever the program has to be tested again (for example, after correcting an error or making an improvement), the test cases must be reinvented. More often than not, since this reinvention requires a considerable amount of work, people tend to avoid it. Therefore, the retest of the program is rarely as rigorous as the original test, meaning that if the modification causes a previously functional part of the program to fail, this error often goes undetected. Saving test cases and running them again after changes to other components of the program is known as *regression testing*.

Principle 8: Do not plan a testing effort under the tacit assumption that no errors will be found.

This is a mistake project managers often make and is a sign of the use of the incorrect definition of testing—that is, the assumption that testing is the process of showing that the program functions correctly. Once again, the definition of testing is the process of executing a program with the intent of finding errors.

Principle 9: The probability of the existence of more errors in a section of a program is proportional to the number of errors already found in that section.

At first glance it makes little sense, but it is a phenomenon present in many programs. For instance, if a program consists of two modules, classes, or subroutines A and B, and five errors have been found in module A and only one error has been found in module B, and if module A has not been purposely subjected to a more rigorous test, then this principle tells us that the likelihood of more errors in module A is greater than the likelihood of more errors in module B.

Another way of stating this principle is to say that errors tend to come in clusters and that, in the typical program, some sections seem to be much more prone to errors than other sections, although nobody has supplied a good explanation of why this occurs. The phenomenon is useful in that it gives us insight or feedback in the testing process. If a particular section of a program seems to be much more prone to errors than other sections, then this phenomenon tells us that, in terms of yield on our testing investment, additional testing efforts are best focused against this error-prone section.

Principle 10: Testing is an extremely creative and intellectually challenging task.

It is probably true that the creativity required in testing a large program exceeds the creativity required in designing that program. We already have seen that it is impossible to test a program sufficiently to guarantee the absence of all errors. Methodologies discussed later in this book let you develop a reasonable set of test cases for a program, but these methodologies still require a significant amount of creativity.